GUÍA PRÁCTICA FORMATIVA: CONSTRUCCIÓN GUIADA DE PIPELINE CI/CD EN JENKINS

Ingeniería de Software V — Semana 10 (Período 2026-2)

Etapas de Pipeline: Pruebas (JaCoCo), Validación SAST (SonarQube), Publicación (Nexus) y Despliegue (Docker)

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS | **DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS INTELIGENTES**

INFORMACIÓN DE LA ACTIVIDAD — IDENTIFICACIÓN

CÓDIGO:	09776 - TIC	**PERÍODO:**	202620 (Semestre 2026-2)
NRC / GRUPO:	11005 / Grupo 001	**MODALIDAD:**	Taller Formativo Guiado en Laboratorio
POLÍTICA IAG:	Nivel 3: Colaboración Asistida	**ENTORNO:**	Estaciones de Trabajo Sala 205M
ESTUDIANTE 1:	_____	**CÓDIGO:**	_____
ESTUDIANTE 2:	_____	**CÓDIGO:**	_____

1 Contexto de Ingeniería: El Ciclo Completo de CI/CD

1.1 El Reto DevOps: De la Integración Continua al Despliegue Automatizado

En las sesiones precedentes, los equipos estructuraron la infraestructura base de DevOps: servidores Jenkins Controller-Agent, webhooks reactivos (Smee.io) y repositorios de artefactos inmutables (Sonatype Nexus). Sin embargo, en un flujo profesional de ingeniería de software, un commit no puede desplegarse en producción sin superar una batería rigurosa de **compuertas de calidad automáticas (Quality Gates)**.

En esta práctica formativa de la **Semana 10**, construiremos paso a paso un pipeline declarativo (`Jenkinsfile`) que consolida las **cuatro etapas cardinales** de la entrega continua:

1. **Etapa 1 — Pruebas & Cobertura (Test Stage):** Compilación, ejecución de pruebas unitarias/integración e instrumentación de cobertura con **JaCoCo** (*Line Coverage* y *Branch Coverage*), reportando los resultados a Jenkins.
2. **Etapa 2 — Validación SAST & Compuerta de Calidad (Sonar Stage):** Análisis estático de código con **SonarQube Server**, auditoría de seguridad contra defectos inyectados (CWE-330 y CWE-476), y suspensión inteligente con `waitForQualityGate()` a la espera del Webhook reactivo.
3. **Etapa 3 — Publicación de Artefactos Inmutables (Publish Stage):** Gobernanza de ramas (`when { branch 'main' }`): empaquetamiento del binario JAR verificado hacia Nexus y construcción de la imagen Docker etiquetada con trazabilidad inmutable (`product-api:${BUILD_NUMBER}`).
4. **Etapa 4 — Despliegue Continuo & Verificación (Deploy Stage):** Despliegue automatizado del contenedor del microservicio en el entorno de ejecución local y validación de salud (*Healthcheck / Smoke Test*) con sondeo HTTP antes de certificar el release.

Componente	Tecnología & Stack	Función en el Pipeline
Microservicio	Spring Boot 3.2, Java 17, Maven	`product-api` (CRUD comercial)
Calidad & SAST	SonarQube Community 10.x + JaCoCo	Detección de bugs, hotspots y cobertura $\geq 80\,\%$
Orquestador CI/CD	Jenkins 2.x Declarativo	Orquestación de stages, ejecución y webhook
Almacenamiento	Sonatype Nexus OSS 3.x	Repositorio inmutable para releases aprobados
Despliegue (CD)	Docker Engine	Contenedor en ejecución en puerto 8085

2 Procedimiento Experimental Guiado Paso a Paso

2.1 Fase 1: Preparación del Entorno y SonarQube Server en Docker (25 min)

SonarQube es la plataforma de inspección continua que analiza estáticamente el código fuente (*Static Application Security Testing - SAST*). En esta fase inicial, desplegaremos y configuraremos la instancia de SonarQube en el laboratorio:

a) Verificación de Docker en Sala 205M: Las estaciones de trabajo de la sala 205M están configuradas para que los estudiantes ejecuten Docker sin necesidad de `sudo`. Verifique su acceso ejecutando:

```
docker ps
# El kernel de Ubuntu en las maquinas de la sala ya dispone de vm.max_map_count >= 262144 para Elasticsearch
```

b) Despliegue del Contenedor SonarQube Community Edition: Ejecute el contenedor de SonarQube publicando el puerto web `9000` con volúmenes persistentes:

```
docker run -d --name sonarqube \
  -p 9000:9000 \
  -v sonarqube_data:/opt/sonarqube/data \
  -v sonarqube_extensions:/opt/sonarqube/extensions \
  -v sonarqube_logs:/opt/sonarqube/logs \
  --restart unless-stopped \
  sonarqube:10-community
# O alternativamente desde el repositorio: docker compose up -d sonarqube
```

Monitoree el arranque con `docker logs -f sonarqube` hasta observar el mensaje `SonarQube is operational` (60 a 90 segundos).

c) Primer Ingreso y Cambio de Credenciales: Abra su navegador en `http://localhost:9000`. Inicie sesión con las credenciales por defecto (`admin` / `admin`) y establezca una contraseña administrativa segura (ej: `IcesiDevOps2026!`).

d) Creación del Proyecto y Generación de Token:

- Haga clic en **Create Project** → **Manually**.
- Configure: **Project display name:** `product-api` • **Project key:** `product-api` • **Main branch name:** `main`. Clic en **Set Up**.
- Seleccione la opción **Locally** para generar las credenciales de análisis.
- En **Generate a token**, asigne el nombre `sonarqube-token`, tipo **Project Analysis Token**, y haga clic en **Generate**. Guarde la cadena generada (`sqp_...`).

e) Configuración del Quality Gate Estricto:

- Menú superior **Quality Gates** → clic en **Create**. Nombre: `Icesi-DevOps-Gate`.
- Agregue condiciones sobre **New Code** (Código Nuevo):
  - **Coverage on New Code** $\geq 80{,}0\,\%$ (Bloquea si la cobertura es inferior a 80 %).
  - **New Bugs** $= 0$ y **New Vulnerabilities** $= 0$ (Ratings de confiabilidad y seguridad = A).
  - **Security Hotspots Reviewed** $= 100{,}0\,\%$.
  - **Duplicated Lines on New Code** $< 3{,}0\,\%$.
- En la pestaña **Projects**, asócielo a `product-api` o establézcalo como **Set as Default**.

f) Configuración del Webhook hacia Jenkins: SonarQube calcula métricas asíncronamente en su *Compute Engine*. Para que Jenkins no quede en espera pasiva consumiendo ciclos, SonarQube le avisa vía Webhook cuando termina:

- En SonarQube: **Administration** → **Configuration** → **Webhooks** → Clic en **Create**.
- **Name:** `Jenkins-Webhook` • **URL:** `http://<jenkins_host>:8080/sonarqube-webhook/` (La barra final `/` es obligatoria). Clic en **Create**.

g) Configuración en Jenkins (Plugin y Credenciales):

- En Jenkins (`:8080`): Verifique tener instalado el plugin **SonarQube Scanner for Jenkins** (identificador `sonar`). Este plugin expone automáticamente el servlet `/sonarqube-webhook/`.
- En **Administrar Jenkins** → **Credentials** → **System** → **Global credentials** → **Add Credentials**: **Kind:** `Secret text` • **Secret:** Pegue el token de SonarQube • **ID:** `sonarqube-token`.
- En **Administrar Jenkins** → **System**: Ubique **SonarQube servers**, marque *Enable injection...*, agregue un servidor: **Name:** `SonarQube-Server` • **Server URL:** `http://<sonarqube_host>:9000` • **Server auth token:** Seleccione `sonarqube-token`. Guarde los cambios.

2.2 Fase 2: Construcción Guiada — Etapa 1: Test & Cobertura con JaCoCo (25 min)

El primer paso en todo pipeline es la verificación del código fuente. No tiene sentido compilar un contenedor si las pruebas no pasan:

a) El Agente Contenerizado: Para evitar instalar JDK ni Maven en el host del servidor Jenkins, usaremos un agente efímero de Docker:

```
agent {
  docker {
    image 'maven:3.9.6-eclipse-temurin-17'
    args '-v /root/.m2:/root/.m2' // Monta la cache de Maven para acelerar descargas
  }
}
```

b) Cómo se redacta la etapa de pruebas en el Jenkinsfile: La meta `clean verify` compila el proyecto y corre las pruebas unitarias y de integración. El plugin de JaCoCo en `pom.xml` instrumenta el bytecode y genera `target/site/jacoco/jacoco.xml`:

```
stage('Pruebas & Cobertura JaCoCo') {
  steps {
    sh 'mvn clean verify'
  }
  post {
    always {
      // Publica los resultados graficos de las pruebas en la interfaz de Jenkins
      junit '**/target/surefire-reports/*.xml'
    }
  }
}
```

Fundamento pedagógico: El bloque `post { always { ... } }` garantiza que, incluso si una prueba falla, Jenkins parsea el XML de JUnit y nos muestra exactamente cuál aserción falló en lugar de un log crudo.

2.3 Fase 3: Construcción Guiada — Etapa 2: Validación SAST y Quality Gate (30 min)

Una vez superadas las pruebas, el código debe ser auditado estáticamente para verificar que no introduce vulnerabilidades, code smells ni caídas de cobertura:

a) El Envoltorio `withSonarQubeEnv`: Inyecta la URL de SonarQube y el token autenticado sin exponer secretos en el script:

```
stage('SonarQube SAST Scan') {
  steps {
    withSonarQubeEnv('SonarQube-Server') {
      sh '''
        mvn sonar:sonar \
          -Dsonar.projectKey=product-api \
          -Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
      '''
    }
  }
}
```

b) La Compuerta Bloqueante (`waitForQualityGate`): SonarQube encola el análisis y le entrega a Maven un identificador único de tarea (`ceTaskId`), almacenado en `target/sonar/report-task.txt`. La directiva `waitForQualityGate()` lee ese archivo y pone a dormir el pipeline. Cuando SonarQube concluye, su Webhook dispara una llamada POST a Jenkins, despertando el build:

```
stage('Quality Gate') {
  steps {
    timeout(time: 5, unit: 'MINUTES') {
      script {
        def qg = waitForQualityGate()
        if (qg.status != 'OK') {
          error "PIPELINE ABORTADO: Quality Gate fallo con veredicto: ${qg.status}"
        }
        echo "Quality Gate APROBADO: Cumple estandares de seguridad y cobertura."
      }
    }
  }
}
```

c) Ejercicio Práctico en Sala: Observando el Fallo y Remediando Defectos: El código inicial de `product-api` contiene 2 defectos deliberados:

- *Security Hotspot (CWE-330):* En `ProductSkuGenerator.java`, uso de `java.util.Random` (pseudoaleatorio débil). **Solución:** Reemplazar por `java.security.SecureRandom`.
- *Bug / Null Pointer (CWE-476):* En `ProductService.java`, desreferencia de `Optional.get()` sin validar. **Solución:** Utilizar `.orElseThrow(() ->new ResourceNotFoundExcep`

Ejecuten el pipeline: verifiquen que SonarQube marca **FAILED (Rojo)** y el stage aborta el build. Luego, apliquen los cambios, suban el commit y comprueben que pasa a **PASSED (Verde)**.

2.4 Fase 4: Construcción Guiada — Etapa 3: Publicación en Nexus e Imagen Docker (20 min)

Una vez que el código fue probado y certificado por el Quality Gate, procedemos a generar el **artefacto inmutable**:

a) Gobernanza de Ramas (`when { branch 'main' }`): Un principio innegociable de DevOps es que las ramas de desarrollo (*feature branches*) solo construyen y validan; **nunca** publican artefactos oficiales en Nexus ni despliegan en producción.

b) Publicación del Binario JAR en Nexus: Se publica el empaquetado `.jar` inmutable en el repositorio Sonatype Nexus:

```
stage('Publicacion en Nexus') {
  when { branch 'main' }
  steps {
    echo "Publicando binario inmutable en Sonatype Nexus..."
    sh 'mvn deploy -DskipTests'
  }
}
```

c) Construcción de la Imagen Docker del Microservicio: Para desplegar la aplicación, definimos un `Dockerfile` liviano y multi-arquitectura en `codigo_base/backend/Dockerfile`:

```
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY target/product-api-1.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

En el pipeline, etiquetamos la imagen con el número único de build de Jenkins (`${BUILD_NUMBER}`) para garantizar total inmutabilidad y auditoría:

```
stage('Empaquetar Imagen Docker') {
  when { branch 'main' }
  steps {
    sh "docker build -t product-api:${BUILD_NUMBER} -t product-api:latest backend/"
  }
}
```

2.5 Fase 5: Construcción Guiada — Etapa 4: Despliegue Continuo (CD) y Healthcheck (30 min)

Esta es la transición hacia la **Entrega y Despliegue Continuo (CD)**. El artefacto certificado se pone en operación de forma automática:

a) Estrategia de Despliegue Idempotente: El pipeline debe ser capaz de ejecutarse repetidamente sin fallar. Por ello, si una versión anterior del contenedor está corriendo, la detenemos y removemos antes de levantar la nueva versión en el puerto `8085`:

```
docker stop product-api-running || true
docker rm product-api-running || true
docker run -d --name product-api-running -p 8085:8080 product-api:${BUILD_NUMBER}
```

b) Validación de Salud del Despliegue (*Smoke Test* / *Healthcheck*): Un error común en pipelines novatos es considerar que un despliegue fue exitoso sólo porque `docker run` retornó 0. Si la aplicación falla al iniciar (ej: error en Spring Context o puerto ocupado), el contenedor morirá a los 3 segundos. El pipeline debe realizar un sondeo HTTP activo:

```
stage('Despliegue Continuo (CD)') {
  when { branch 'main' }
  steps {
    echo "Desplegando version product-api:${BUILD_NUMBER} en entorno Staging..."
    sh '''
      docker stop product-api-running || true
      docker rm product-api-running || true
      docker run -d --name product-api-running -p 8085:8080 product-api:${BUILD_NUMBER}
    '''
    echo "Ejecutando Smoke Test / Healthcheck de verificacion..."
    sh '''
      # Sondeo reactivo: espera hasta 30 segundos a que Spring Boot inicie
      for i in {1..10}; do
        if curl -s -f http://localhost:8085/api/products; then
          echo "DESPLIEGUE EXITOSO: El microservicio responde HTTP 200 en puerto 8085."
          exit 0
        fi
        echo "Esperando arranque del servicio... (intento $i de 10)"
        sleep 3
      done
      echo "ERROR CRITICO: El microservicio no respondio a tiempo tras el despliegue."
      exit 1
    '''
  }
}
```

2.6 Fase 6: El Jenkinsfile Declarativo Completo y Consolidado

A continuación se presenta la arquitectura del archivo `Jenkinsfile` unificado que los estudiantes deben construir y probar en la raíz de su repositorio:

```
pipeline {
  agent {
    docker {
      image 'maven:3.9.6-eclipse-temurin-17'
      args '-v /root/.m2:/root/.m2 -v /var/run/docker.sock:/var/run/docker.sock'
    }
  }

  options {
    timeout(time: 20, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '10'))
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }

    // ====================================================================
    // ETAPA 1: TEST & COBERTURA
    // ====================================================================
    stage('1. Test & Cobertura JaCoCo') {
      steps {
        echo "Ejecutando pruebas unitarias y recolectando metricas JaCoCo..."
        sh 'mvn clean verify'
      }
      post {
        always {
          junit '**/target/surefire-reports/*.xml'
        }
      }
    }

    // ====================================================================
    // ETAPA 2: VALIDACION SAST & QUALITY GATE CON SONARQUBE
    // ====================================================================
    stage('2. SonarQube SAST Scan') {
      steps {
        echo "Despachando analisis estatico SAST al servidor SonarQube..."
        withSonarQubeEnv('SonarQube-Server') {
          sh '''
            mvn sonar:sonar \
              -Dsonar.projectKey=product-api \
              -Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
          '''
        }
      }
    }

    stage('2.1. Quality Gate Enforcement') {
      steps {
        echo "Esperando veredicto del Quality Gate via Webhook de SonarQube..."
        timeout(time: 5, unit: 'MINUTES') {
          script {
            def qg = waitForQualityGate()
            if (qg.status != 'OK') {
              error "PIPELINE ABORTADO: Quality Gate rechazo el build (${qg.status})"
            }
            echo "Quality Gate APROBADO: Cumple cobertura >= 80% y 0 vulnerabilidades."
          }
        }
      }
    }

    // ====================================================================
    // ETAPA 3: PUBLICACION EN NEXUS E IMAGEN DOCKER
    // ====================================================================
    stage('3. Publicacion en Nexus') {
      when { branch 'main' }
      steps {
        echo "Publicando binario inmutable product-api-1.0.0.jar en Sonatype Nexus..."
        sh 'mvn deploy -DskipTests'
      }
    }

    stage('3.1. Empaquetar Imagen Docker') {
      when { branch 'main' }
```

```
      steps {
        echo "Construyendo imagen Docker inmutable product-api:${BUILD_NUMBER}..."
        sh "docker build -t product-api:${BUILD_NUMBER} -t product-api:latest backend/"
      }
    }

    // ====================================================================
    // ETAPA 4: DESPLIEGUE CONTINUO (CD) & HEALTHCHECK
    // ====================================================================
    stage('4. Despliegue Continuo (CD)') {
      when { branch 'main' }
      steps {
        echo "Desplegando contenedor en puerto 8085..."
        sh '''
          docker stop product-api-running || true
          docker rm product-api-running || true
          docker run -d --name product-api-running -p 8085:8080 product-api:${BUILD_NUMBER}
        '''
        echo "Validando arranque del microservicio con Smoke Test..."
        sh '''
          for i in {1..10}; do
            if curl -s -f http://localhost:8085/api/products; then
              echo ">>> EXITO: Servicio desplegado y operativo en http://localhost:8085 <<<"
              exit 0
            fi
            sleep 3
          done
          echo ">>> ERROR: El servicio no inicio correctamente. <<<"
          exit 1
        '''
      }
    }
  }

  post {
    success {
      echo "CICLO CI/CD COMPLETADO CON EXITO: Test, Sonar, Nexus y Despliegue conformes."
    }
    failure {
      echo "ALERTA: Pipeline fallido. Revise pruebas, Quality Gate o logs del contenedor."
    }
  }
}
```

3 Cuestionario de Análisis Técnico y Reflexión DevOps

Responda en su bitácora de laboratorio o reporte de equipo las siguientes preguntas de reflexión ingenieril:

1. **Gobernanza y Compuertas de Bloqueo:** ¿Por qué es una pésima práctica de ingeniería permitir que la etapa de *Publicación a Nexus* o *Despliegue* se ejecute antes de evaluar el *Quality Gate*? Describa el impacto técnico y de negocio de permitir que un binario con vulnerabilidades llegue al repositorio inmutable o a producción.

2. **Mecanismo de Webhooks vs. Polling Activo:** Explique detalladamente cómo interactúan el archivo `report-task.txt`, el identificador `ceTaskId`, la instrucción `waitForQualityGate()` y el endpoint `/sonarqube-webhook/`. ¿Por qué esta arquitectura basada en eventos reactivos es superior al sondeo periódico (*polling*) en un clúster de CI/CD?

3. **Resiliencia y Circuit Breaking con `timeout`:** Si ya tenemos un Webhook reactivo configurado entre SonarQube y Jenkins, ¿por qué es mandatorio rodear la llamada con `timeout(time: 5, unit: 'MINUTES')`? Explique qué sucedería en el servidor Jenkins ante un fallo de red o un reinicio no programado de SonarQube si no existiera dicho límite temporal (*Zombie Builds*).

4. **Falso Positivo en Despliegues y el Rol del Healthcheck:** ¿Por qué no es suficiente con que el comando `docker run` retorne código de salida `0` para declarar exitosa la etapa de despliegue? ¿Qué tipo de fallos en aplicaciones Spring Boot sólo se manifiestan segundos después del arranque del contenedor y cómo el *Smoke Test* con `curl` protege la estabilidad del entorno?

4 Lista de Chequeo de Autoverificación Formativa (Checklist de Éxito)

Utilicen esta lista de chequeo para verificar que su pipeline cumple todos los estándares formativos:

Hito	Componente	Criterio Técnico Observable	Estado
1	Servidor SonarQube	Contenedor en `:9000`, credenciales cambiadas, proyecto creado y token registrado.	[ ] Superado
2	Webhook Integrado	Webhook emisor en SonarQube hacia `http://<host>:8080/sonarqube-webhook/`.	[ ] Superado
3	Etapa 1: Test JaCoCo	`mvn clean verify` exitoso; XML de JUnit visible en gráfica de Jenkins.	[ ] Superado
4	Etapa 2: Sonar Scan	Escaneo ejecutado con `withSonarQubeEnv`; reporte subido exitosamente.	[ ] Superado
5	Etapa 2.1: Quality Gate	Pipeline se pausa y reanuda con webhook; aborta en rojo ante defectos y aprueba en verde.	[ ] Superado
6	Etapa 3: Publicación	Solo corre en `main`; JAR publicado en Nexus e imagen Docker etiquetada.	[ ] Superado
7	Etapa 4: Despliegue	Contenedor `product-api-running` en puerto `8085`; Healthcheck responde HTTP 200.	[ ] Superado